

OXYPC INVENTORY LIFECYCLE

PROCESS

Business Process & Rules

How a device moves through every stage, and the rules each team follows.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Business Process & Rules

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Device Lifecycle State Machine · Part B — Business Rules & SOPs

PART A

Device Lifecycle State Machine

1. Purpose

This document describes how one device (one **Tag Number**, stored as `devices.barcode`) moves through the OxyPC Inventory Lifecycle app. It is written from the code, not from the business view. Where the two differ, the gap is listed in section 9.

Use it to:

- see every stage value the app stores in `devices.current_stage`;
- find which screen and endpoint moves a tag from one stage to another;
- check the guards (what must be true) and side effects (WorkIDs, part requests, audit rows) of each move;
- understand the smaller state machines that run beside the main one (repair status, cosmetic chain, Final QC holds, part requests, PNA, returns, credit notes, block tags).

Related: Business Rules & SOPs · Role & Permission Matrix.

1.1 How stage moves are recorded

Record	What it holds	Where
`devices.current_stage`	The one current stage (enum `DeviceStage`)	`models/device.py::Device`
`stage_movements`	One row per move: from, to, who, when, note. `exited_at` is stamped on the previous open row when the tag leaves a stage	`models/device.py::StageMovement`
`work_orders`	One row per assignment to a person (the **WorkID**). Status `pending` → `in_progress` → `completed`	`models/work_order.py::WorkOrder`
`audit_logs`	Business audit rows written by `services/audit_engine.py::audit` (not every move writes one)	`services/audit_engine.py`
`allowed_transitions`	Admin-editable table of legal moves. Only some endpoints check it (see 1.2)	`models/stage_control.py`, `routers/stage_control.py`

WorkID formats (from `routers/transfers.py::_gen_work_id` and `routers/repair.py::_gen_prefixed_work_id`):

WorkID	Meaning	`work_orders.stage` code
12-digit number	Normal assignment (pick, bucket assign, cosmetic move, Final QC decision, Stress result)	`l1`, `qc`, `strs`, `recv`, `clean`, `putty`, `drysd`, `mask`, `paint`, `water`, `comp`, `fqc`, `asgn`, or first 5 letters of the stage

WorkID	Meaning	`work_orders.stage` code
`L3L4-0000001`	L3/L4 repair request	`l3`
`STRS-0000001`	Hand-off from L1/L2 Mark Complete to Stress Test	`qc`
`L1L2-…`	Legacy L1/L2 WorkIDs (still closed by `_close_l1l2_work_orders`)	`l1` / `l2`

1.2 The Control Engine

`services/control_engine.py` has four checks:

Function	Rule	Used by
`assert_device_in_stage`	Tag must be in the expected stage, else HTTP 409 "STAGE CONFLICT"	`routers/repair.py::start_repair`, `complete_repair`; `routers/qc.py::qc_submit`
`validate_transition`	Move must exist in `allowed_transitions`, else HTTP 403. Admin bypasses	`routers/repair.py::start_repair`, `complete_repair`, `move_device`; `routers/qc.py::qc_submit`; `routers/grn.py::grn_bulk_move_to_trc`; `routers/iqc.py::iqc_bulk_apply_grade_type`; `routers/bulk_upload.py::approve_duplicate_move`; `routers/api_v1/devices.py::move_device_stage`
`validate_sale_allowed`	Refuses a sale only when `devices.is_blocked` is true. Since 19 Oct 2026 a sale is allowed **from any stage**	`routers/sales.py::create_sale`
`validate_repair_level`	L2 needs an L1 RepairJob; L3 needs an L2 RepairJob	`routers/repair.py::start_repair`

Most dedicated workflow screens (L1/L2 Mark Complete, Stress Test, Cosmetic, Final QC, buckets, returns) **do not** call `validate_transition`. Their own code decides the next stage.

2. Stage reference

Stage values come from `models/device.py::DeviceStage`. Display names come from `STAGE_LABELS`. Badge colours come from `STAGE_COLORS` (Bootstrap colour names; `teal` and `purple` are custom).

Owner role = the business persona that works the stage. The code role in brackets is the built-in `UserRole` that the code actually checks.

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
1	`grn`	GRN Receipt	warning	IQC Inspector (any signed-in user)	Register tag on lot: `routers/stock.py::register_device_barcode`, `register_bulk_csv`	Advance GRN to IQC → `iqc`

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
2	`iqc`	IQC	secondary	IQC Inspector (`iqc_inspector`)	IQC form without GRN no. (`routers/iqc.py::iqc_create`); OxyQC agent (`routers/iqc_api.py::submit_iqc`); GRN Records "Sending for Selling"; return approved; External Tag return creates new tag	Map GRN / Validate GRN → `stock_in` ; Customise move
3	`stock_in`	Stock In	info	Inventory Manager (`inventory_manager`)	GRN mapped/validated; IQC form with GRN no.; bucket create; Back to Inventory from repair; return submitted; manual moves	Move to Production → `trc_production` ; Move to L1/L2 → `l1` ; Customise / manual moves
4	`trc_production`	TRC Production	info	Production Manager (`inventory_manager` , or custom role `production_manager`)	Move to TRC / Move bucket to Production; Final QC Fail (Hardware, no engineer found); Change Engineer	Assign / Bulk Assign / Change Engineer → `l1` , `qc_check` , `cosmetic_received`
5	`l1`	L1/L2 Repair	warning	Repair Engineer L1/L2 (`l1_engineer` , `l2_engineer`)	Move to L1/L2; bucket / device assign; Stress fail; Cosmetic fail; Final QC fail (Hardware); L3/L4 complete or scrap decision	Request to L3/L4 → `l3` ; Mark Complete → `qc_check` ; Back to Inventory → `stock_in` or `scrap_for_sale` ; Scrap → `scrapped`
6	`l2`	L2 Repair	warning	Legacy — no page	None today (retired; kept for old rows)	Legacy `complete_repair` moves
7	`l3`	L3/L4 Repair	danger	Repair Engineer L3/L4 (`l3_engineer`)	Request to L3/L4 from L1/L2; Change Engineer	L3/L4 Complete or Scrap → back to `l1`
8	`qc_check`	Stress Test	primary	QC Inspector (`qc_inspector`)	L1/L2 Mark Complete; bucket assign "Stress Test"; Final QC fail (Software); Change Engineer	Complete → `cosmetic_received` ; Fail → `l1`
9	`cosmetic_received`	Cosmetic Received	teal	Cosmetic Lead (`cosmetic_manager`)	Stress Complete; bucket assign "Cosmetic Repair"; Final QC fail (Cosmetic); Send to Cosmetic	Move → `cleaning` ; Skip → `final_qc` ; Fail → `l1`
10	`cleaning`	Cleaning	teal	Cosmetic Lead / stage user	Move from Cosmetic Received; bulk Assign	Move → `putty` ; Skip → `final_qc` ; Fail → `l1`
11	`putty`	Putty	teal	Cosmetic Lead / stage user	Move from Cleaning	Move → `dry_sanding` ; Skip → `final_qc`
12	`dry_sanding`	Dry Sanding	teal	Cosmetic Lead / stage user	Move from Putty	Move → `masking` ; Skip → `final_qc`
13	`masking`	Masking	teal	Cosmetic Lead / stage user	Move from Dry Sanding	Move → `painting` ; Skip → `final_qc`

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
14	`painting`	Painting	teal	Cosmetic Lead / stage user	Move from Masking	Move → `water_sanding`; Skip → `final_qc`
15	`water_sanding`	Water Sanding	teal	Cosmetic Lead / stage user	Move from Painting	Move → `cosmetic_completed`; Skip → `final_qc`; Fail → `l1`
16	`cosmetic_completed`	Cosmetic Completed	teal	Cosmetic Lead	Move from Water Sanding	Move → `final_qc`; Fail → `l1`
17	`final_qc`	Final QC	purple	Final QC Inspector (any role with `cosmetic_finalqc` edit permission)	From cosmetic chain; Edit (revert) from a hold; Change Engineer	Pass → `final_qc_pass_hold`; Fail → `final_qc_fail_hold`
18	`final_qc_pass_hold`	Final QC — Pass Hold	success	Final QC Inspector → Inventory Manager	Final QC Pass	Move to Inventory → `ready_to_sale`; Edit → `final_qc`
19	`final_qc_fail_hold`	Final QC — Fail Hold	warning	Final QC Inspector	Final QC Fail	Move / Bulk Move → `l1`, `trc_production`, `qc_check` or `cosmetic_received`; Edit → `final_qc`
20	`ready_to_sale`	Ready to Sale	success	Inventory Manager / Sales Executive	Move to Inventory from Pass Hold; ReGRN of a sold tag; Trade Partner test devices; Customise	Sale → `sold`
21	`sold`	Sold	dark	Sales Executive (`sales`, `sales_manager`, `telecaller`)	New Sale / New Tag Sale / Sell from Block; Return Replace Now; Return Stock Complete	Return submitted → `stock_in`; ReGRN → `ready_to_sale`; Sale deleted → previous stage
22	`returned`	Returned	warning	—	**No code path sets this stage today** (see Q2)	Allowed transitions list `iqc`, `scrapped`, `stock_in`
23	`scrapped`	Scrapped	danger	Inventory Manager / Production Manager	Repair "Scrap" or cost auto-scrap (`services/cost_engine.py::auto_scrap_device`); return approved with action Scrap	"Move to Inventory" → `scrap_for_sale`
24	`scrap_for_sale`	Scrap for Sale	dark	Inventory Manager (Tags Scrapped page)	Back to Inventory after L3/L4 scrap; Scrap Products "Move to Inventory"; Customise	None in code (scrap sale does not change stage, see Q6)

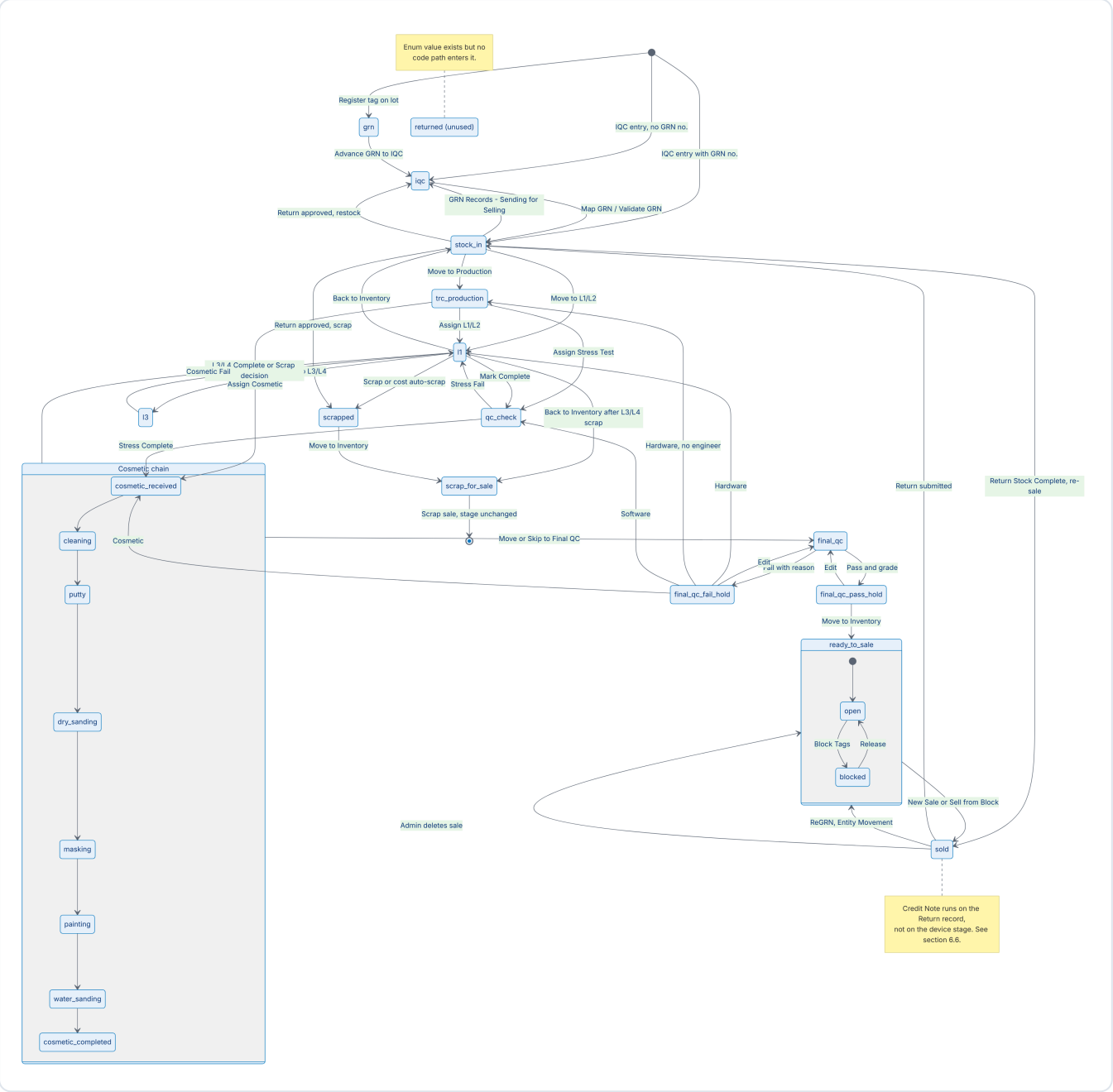
Notes:

- `DROPDOWN_STAGES` (all stages except `l2` and `sold`) feeds every generic "move to any stage" dropdown (`models/device.py`).

- `COSMETIC_STAGES` groups stages 9–16 for dashboard counts.
- `stage_sort_expr` sorts stages in the enum order above.
- `is_blocked`, `return_status`, `tag_return_status`, `final_qc_status`, `l1l2_status` and `l34_status` are **flags beside the stage**, not stages.

3. Lifecycle diagram

The diagram shows the main paths. Generic admin/Customise moves (any stage to any stage) are left out to keep it readable; see transition T40–T43.



4. Transitions

"Role" shows the code guard first, then the business persona. "Any user" means the endpoint only checks that someone is signed in (`get_current_user`).

4.1 Intake

#	From	To	Trigger / screen	Role	Guards	Side effects
T01	(new)	`grn`	Lot → Register Device / Bulk CSV (`routers/stock.py::register_device_barcode` , `register_bulk_csv`)	Any user · IQC Inspector	Lot exists	Device row created
T02	`grn`	`iqc`	Lot → Advance GRN to IQC (`routers/stock.py::advance_grn_to_iqc`)	Any user · IQC Inspector	Only tags at `grn` in that lot	**No StageMovement, no audit** (Q12)
T03	(new)	`iqc` or `stock_in`	IQC form (`routers/iqc.py::iqc_create`)	`admin` , `inventory_manager` , `iqc_inspector` , `sales_manager` · IQC Inspector	Barcode must be new; lot must resolve (blank lot → "Unassigned" lot via `utils/lot_helpers.get_or_create_unassigned_lot`). GRN no. given → `stock_in` , else `iqc`	IQCInspection row; StageMovement "IQC Entry" (+ second row to Stock In if GRN given); `device_price` from lot line item or lot average; audit `DEVICE_IQC_REGISTERED`
T04	(new)	`iqc`	OxyQC agent submit (`routers/iqc_api.py::submit_iqc`), API (`routers/api_v1/iqc.py::register_device`)	API / agent	—	Device + inspection rows
T05	`iqc`	`stock_in`	GRN Post-IQC → Map this GRN (`routers/grn.py::grn_map`)	`admin` , `inventory_manager` · Inventory Manager (business: IQC Inspector)	Only tags at `iqc` move; others just get the GRN no.	`grn_number` set; blank entity → "OxyPC Computers"; StageMovement; default location; GRN Stocked memory; audit `GRN_MAPPED`
T06	`iqc`	`stock_in`	GRN Import → Validate (`routers/grn.py::grn_validate`)	`admin` , `inventory_manager`	GRN linked to a lot; tags of that lot at `iqc`	GRN marked validated; auto-map as T05. No audit row
T07	any (allowed)	`iqc` / `stock_in`	GRN Records → Pending for TRC bulk move (`routers/grn.py::grn_bulk_move_to_trc`)	`admin` , `inventory_manager`	`validate_transition` (admin bypass); failed tags skipped	"Selling" → `iqc` and GRN cleared; "Sold to OxyPC" → `stock_in` ; audit `GRN_BULK_MOVE_TO_TRC`

4.2 Stock and production

#	From	To	Trigger / screen	Role	Guards	Side effects
T08	any	`stock_in`	Create bucket (`routers/buckets.py::create_bucket`)	`admin` , `inventory_manager` · Inventory Manager	Active, not trashed. **No stage guard**	Bucket created or reused by name; location set. **No StageMovement** (Q12)
T09	`stock_in`	`trc_production`	Move to TRC (`routers/stock.py::move_to_trc`)	`admin` , `inventory_manager` , `qc_inspector` , `sales_manager` · Inventory Manager	Tag must be at `stock_in`	StageMovement
T10	`stock_in` (or any, for customer-return buckets)	`trc_production`	Bucket → Move to Production / Assign to Production (`routers/buckets.py::move_bucket_to_trc` , `assign_bucket_to_production` → `_move_bucket_devices_to_trc`)	`admin` , `inventory_manager`	Only `stock_in` tags, unless `bucket.is_customer_return`	Bucket status `trc_pending` , `assigned_to_production` ; StageMovement per tag
T11	`stock_in`	`l1`	Move to L1/L2, single or bulk (`routers/stock.py::move_to_l1l2` , `bulk_move_to_l1l2` → `_move_device_to_l1l2`)	Stock roles · Inventory Manager	Tag must be at `stock_in`	Stale open `l1` WorkOrders closed; repair status reset to New (`reset_repair_cycle_on_entry`); lands **unassigned** (Pick This)
T12	any	`l1` / `qc_check` / `cosmetic_received`	Production Manager → Assign bucket / Assign device / Bulk Assign (`routers/buckets.py::assign_bucket` , `assign_device` , `bulk_assign_devices_l1l2` → `_apply_department_move`)	`admin` , `inventory_manager` (+ custom `production_manager`) · Production Manager	Department must map in `DEPT_TO_STAGE` ("L1/L2 Repair", "Stress Test", "Cosmetic Repair")	StockTransfer (`department_assignment`); open WorkOrders closed (`services/work_order_close.py::close_open_work_orders`); new WorkID or reuse of open unassigned STRS- (`reuse_open_stress_work_order`); notification; status reset on L1/L2 entry
T13	any	`l1`	Bucket → Assign to Engineer (`routers/buckets.py::assign_bucket_to_engineer`)	`admin` , `inventory_manager`	—	StageMovement; status reset; bucket `validated`

#	From	To	Trigger / screen	Role	Guards	Side effects
T14	any active	`l1`, `l3`, `qc_check`, `cosmetic_received`, `final_qc`, `stock_in`, `trc_production`	Change Engineer (`routers/stock.py::change_engineer`)	Stock roles · Production Manager	Target in `CHANGE_ENGINEER_STAGES`; user active. **No source-stage guard**	Close open WorkOrders, new WorkID (or STRS- reuse); notification; audit `change_engineer`
T15	any	`stock_in`	Move to Stock (`routers/stock.py::move_to_stock`)	Stock roles	**No stage guard**	StageMovement; flag cleared by `_clear_flag_on_stock_in`

4.3 Repair (L1/L2 and L3/L4)

#	From	To	Trigger / screen	Role	Guards	Side effects
T16	`l1`	`l1`	Pick This (`routers/repair.py::l1_pick`)	Permission `repair_l1` add · Repair Engineer	Tag at `l1`; picking own tag is a no-op	All open WorkOrders closed; new 12-digit WorkID; same-stage StageMovement; previous engineer notified; audit `L1L2_PICKED`
T17	`l1`	`l1`	Start Repair (`routers/repair.py::l1l2_start`)	Any user	—	`l1l2_status` = "Repair Started"; RepairJob opened; audit
T18	`l1`	`l3`	Request to L3/L4 (`routers/repair.py::request_l3l4`)	Any user · Repair Engineer L1/L2	L3/L4 engineer must exist	`l1l2_status` = "Requested to L3/L4"; optional `repair_notes`; new `L3L4-` WorkID (L1/L2 WorkID left open); notification; audit `REQUEST_L3L4`
T19	`l3`	`l3`	L3/L4 Pick / Start (`routers/repair.py::l3l4_pick`, `l3l4_start`)	`repair_l3` enable (pick) / any user (start)	Tag at `l3` (pick)	Pick: close open WorkIDs, new `L3L4-` WorkID. Start: `l34_status` = "Repair Started"
T20	`l3`	`l1`	L3/L4 Mark Complete (`routers/repair.py::l3l4_complete`)	Any user · Repair Engineer L3/L4	—	`L3L4-` WorkOrders completed; `l34_status` = "Completed"; `l1l2_status` = "Returned from L3/L4"; activity + notes; audit
T21	`l3`	`l1`	L3/L4 Scrap (`routers/repair.py::l3l4_scrap`)	Any user · Repair Engineer L3/L4	Type must be "Normal Scrap" or "Replacement Scrap"	`l34_status` = scrap type; L1/L2 page then shows "Back to Inventory"

#	From	To	Trigger / screen	Role	Guards	Side effects
T22	`l1` (any)	`scrap_for_sale`	Back to Inventory after scrap decision (`routers/repair.py::back_to_production`)	Any user · Repair Engineer	**No stage or status guard**	`grade` = scrap; RepairJobs closed; audit `BACK_TO_INVENTORY` . Tag shows on Tags Scrapped page
T23	`l1` (any)	`qc_check`	L1/L2 Mark Complete (`routers/repair.py::l1l2_complete_to_stress`)	Any user · Repair Engineer	**Blocked if any active PNA part**; no stage guard	Open part requests: `handed_over` → `received` (cost counted), `requested` → `cancelled` ; L1/L2 WorkOrders and RepairJobs closed; `l1l2_status` = "Completed"; new `STRS-` WorkID (engineer optional); audits `PART_AUTO_VERIFIED_ON_COMPLETE` , `PART_AUTO_CANCELLED_ON_COMPLETE` , `L1L2_COMPLETE_TO_STRESS`
T24	`l1`	`l1` / `qc_check` / `stock_in` / `scrapped`	Legacy side panel "Complete Job" (`routers/repair.py::complete_repair`)	Permission `repair_<stage>` edit	`assert_device_in_stage` ; `validate_transition` on some branches	Final status decides: Scrap or cost auto-scrap → `scrapped` ; Escalate → stays `l1` ; PNA → no move; Back to Inventory → `stock_in` ; Completed/OK → `qc_check` (no STRS- WorkID). Writes RepairAttempt, parts consumption, `REPAIR_COMPLETE` audit
T25	`stock_in` / `l1`	`l1`	Legacy Start Repair (`routers/repair.py::start_repair`)	Permission `repair_l1` add	Stage check, repair-level check, transition check	RepairJob opened

4.4 Stress Test, cosmetic and Final QC

#	From	To	Trigger / screen	Role	Guards	Side effects
T26	`qc_check` (any)	`l1`	Stress Test → Fail (`routers/stress_api.py::stress_fail_assign`)	Any user · QC Inspector	Engineer must have role `l1_engineer` or `l2_engineer` . **No stage guard**	`stress_notes` = failed checklist items + note; status reset; checklist result + PDF saved; completed `strs` WorkID for the tester; new `l1` WorkID for engineer; notification; audit `STRESS_FAIL_ASSIGNED`
T27	`qc_check` (any)	`cosmetic_received`	Stress Test → Complete (`routers/stress_api.py::stress_complete_to_paint`)	Any user · QC Inspector	Optional engineer must be `qc_inspector` , `inventory_manager` , `sales_manager` or `cosmetic_manager`	`stress_notes` cleared; checklist + PDF; completed `strs` WorkID; new `recv` WorkID (may be unassigned); audit `STRESS_COMPLETE_TO_PAINT`

#	From	To	Trigger / screen	Role	Guards	Side effects
T28	`qc_check`	`cosmetic_received` or `l1` / `l2` / `l3`	Legacy QC form (`routers/qc.py::qc_submit`)	`qc_check` add permission	`assert_device_in_stage` ; `validate_transition`	QCCheck row with scores; pass sets grade; 3rd fail notes "recommend scrap"
T29	any	`cosmetic_received`	Send to Cosmetic (`routers/cosmetic.py::send_to_cosmetic`)	Cosmetic roles + `cosmetic_received` add	**No stage guard**	StageMovement only
T30	cosmetic stage *n*	stage *n*+1	Cosmetic page → Move (`routers/cosmetic.py::advance_stage`)	Cosmetic roles + per-stage edit permission (`PERM_MODULE_BY_STAGE`) · Cosmetic Lead	Stage must be in `NEXT_COSMETIC` . Assignee required unless landing on `final_qc` or `cosmetic_completed` ; Flow Data can auto-pick (`_resolve_flow_next_user`)	Origin stage's pending WorkOrder completed; new WorkID for assignee; notification. **No audit row**
T31	any cosmetic stage	`final_qc`	Skip cosmetic (`advance_stage` with `target=final_qc`)	As T30	Tag in `COSMETIC_PIPELINE`	`bucket_id` cleared on arrival at Final QC
T32	`cosmetic_received`	`cleaning`	Bulk Assign (`routers/cosmetic.py::bulk_assign`)	`admin` or `cosmetic_manager` only	Tag in `BULK_ASSIGN_STAGES`	New WorkID per tag. Other cosmetic stages: reassignment only, no move
T33	`cosmetic_received` , `cleaning` , `water_sanding` , `cosmetic_completed`	`l1`	Cosmetic → Fail (`routers/cosmetic.py::cosmetic_fail_assign`)	Per-stage edit permission · Cosmetic Lead	Only these 4 stages; engineer must be `l1_engineer` / `l2_engineer`	`repair_notes` = " Failed — ..."; status reset; `l1` WorkID; notification; audit `COSMETIC_FAIL_ASSIGNED`
T34	`final_qc`	`final_qc_pass_hold`	Final QC → Pass (`routers/cosmetic.py::advance_stage`)	`cosmetic_finalqc` edit · Final QC Inspector	—	Completed `fqc` WorkID for the inspector; grade set; touch type; spec corrections; verified RAM/HDD changes written (`apply_spec_changes_on_pass`); **price finalised once** (base + parts consumed + changed parts); bucket by name
T35	`final_qc`	`final_qc_fail_hold`	Final QC → Fail (`advance_stage`)	`cosmetic_finalqc` edit	—	`final_qc_status` = fail; `fqc_failure_reason` , `fqc_final_notes` ; bucket fail reason/engineer recorded
T36	Pass/Fail Hold	`final_qc`	Edit (`routers/cosmetic.py::fqc_edit_revert`)	`cosmetic_finalqc` edit	Tag must be in a hold	Decision fields cleared

#	From	To	Trigger / screen	Role	Guards	Side effects
T37	`final_qc_pass_hold`	`ready_to_sale`	Devices Passed → Move to Inventory (`routers/cosmetic.py::fqc_move_passed`)	`cosmetic_finalqc` edit · Final QC Inspector → Inventory Manager	Tag must be at Pass Hold	StageMovement; default location
T38	`final_qc_fail_hold`	`l1` / `trc_production` / `qc_check` / `cosmetic_received`	Devices Failed → Move / Bulk Move (`routers/cosmetic.py::fqc_move_failed`)	`cosmetic_finalqc` edit	Tag at Fail Hold and reason is Hardware, Software or Cosmetic	Routed by `FAIL_REASON_TO_DEPT` ; to last engineer at that step (`_resolve_fail_engineer`). Hardware with no engineer → `trc_production` . Uses `_apply_department_move` (T12 side effects)

4.5 Sales, returns and scrap

#	From	To	Trigger / screen	Role	Guards	Side effects
T39	**any stage**	`sold`	New Sale / New Tag Sale / Ready to Sale bulk / Sell from Block (`routers/sales.py::create_sale`)	`admin` , `sales` , `sales_manager` , `telecaller` + `sales` add · Sales Executive	Tag entity = seller's `User.entity` (seller must have one); not blocked unless sold from its own block (`validate_sale_allowed`)	Sale row `SALE-nnnn` ; company snapshot; warranty expiry; sale invoice no. replaces tag invoice no.; block item released with note "Sold — SALE-..."; below-cost warning (not blocking); audit `SALE_CREATED` ; event `SALE_COMPLETED`
T40	any	any	Customise modal → Move to Stage (`routers/iqc.py::iqc_bulk_apply_grade_type`)	IQC roles + `iqc` or `devices` edit	`sold` refused. `validate_transition` skipped for admin and when posted from `/devices` or `/stock`	Status reset on L1/L2 entry; L1/L2 or L3/L4 WorkOrders closed on exit; spec changes past Final QC
T41	any	any (allowed)	Manual move (`routers/repair.py::move_device`)	Any user	`validate_transition` (admin bypass). **Does not refuse `sold`**	Closes `L3L4-` WorkOrders when leaving `l3` ; audit `STAGE_MOVED`
T42	any	any (allowed)	API v1 (`routers/api_v1/devices.py::move_device_stage`)	API key	Transition must exist	StageMovement by `api_key:<name>`
T43	any	any (allowed)	Bulk upload duplicate → Approve Move (`routers/bulk_upload.py::approve_duplicate_move`)	Bulk upload roles	`validate_transition`	audit `STAGE_MOVED`

#	From	To	Trigger / screen	Role	Guards	Side effects
T44	`sold` (or any with a sale)	`stock_in`	Return New → Internal Tag (`routers/sales.py::process_return`)	Sales roles + `returns` add · Sales Executive	A sale must exist (or tag at `sold` → placeholder sale); one return per sale; reason, condition, received-by required	Return row `pending` ; `return_status` = true; `tag_return_status` = "Return for Repair" or "Replaced by "; audit `RETURN_SUBMITTED`
T45	`ready_to_sale`	`sold`	Return New → Replace Now (`process_return`)	As T44	Replacement tag must be `ready_to_sale` and not the same tag	₹0 Sale with original customer; `replaced` = "Replaced with "; customer invoice no. copied; audit `RETURN_REPLACEMENT_SOLD`
T46	any	`iqc` / `scrapped`	Pending Returns → Approve (`routers/sales.py::approve_return`)	`admin` , `sales_manager` only	Return must be `pending`	Restock → `iqc` ; Scrap → `scrapped` ; audit `RETURN_APPROVED`
T47	—	—	Pending Returns → Reject (`reject_return`)	`admin` , `sales_manager`	Return must be `pending`	Status `rejected` . **Tag stays at `stock_in`** (Q2)
T48	(new)	`iqc`	Return New → External Tag (`process_external_return`)	As T44	Reason, condition, received-by required	New device in lot `EXT-RETURNS` if tag unknown; placeholder ₹0 sale; pending Return with paid-repair amount
T49	any (returned tag)	`sold`	Return Stock → Complete (`routers/stock.py::return_stock_complete`)	Stock roles · Inventory Manager	A return and its original sale must exist	New Sale with original customer and warranty days; `return_status` = false; audit; event `SALE_COMPLETED`
T50	`sold`	previous stage	Sales list → Delete Selected (`routers/sales.py::bulk_delete_sales`)	`admin` only	Referenced sales (credit notes, receipts, care) skipped	Audit `SALE_DELETED` before delete; tag returns to the stage it was sold from
T51	`sold`	`ready_to_sale`	Entity Movement → ReGRN (`routers/entity_movement.py`)	`entity_movement` permission	Tag not trashed	New GRN/invoice/entity; location; audit `REGRN_TAG`
T52	any	`scrapped`	Cost engine auto-scrap (`services/cost_engine.py::auto_scrap_device` , from `complete_repair`)	System	Total cost ≥ expected sale value	AuditLog `AUTO_SCRAP`

#	From	To	Trigger / screen	Role	Guards	Side effects
T53	any	`scrap_for_sale`	Scrap Products from Repair Line → Move to Inventory (`routers/stock.py::scrap_move_to_inventory`)	Stock roles	**No stage guard**	StageMovement only
T54	`ready_to_sale`	`ready_to_sale` (blocked)	Block Tags → New Block (`routers/block_tags.py::create_block`)	`block_tags` add · Inventory Manager, Sales Executive	Not already blocked. Ready-to-Sale check is in the search box only (Q11)	`is_blocked` = true; BlockRecord `DDMMYYYY-CONTAINER-0001` ; `assigned_to_user_id` = Block For user. **No audit**
T55	blocked	open	Block Tags → Release / Bulk Release (`_release_items`)	`block_tags` edit	Item still `blocked`	`is_blocked` = false; item `released`

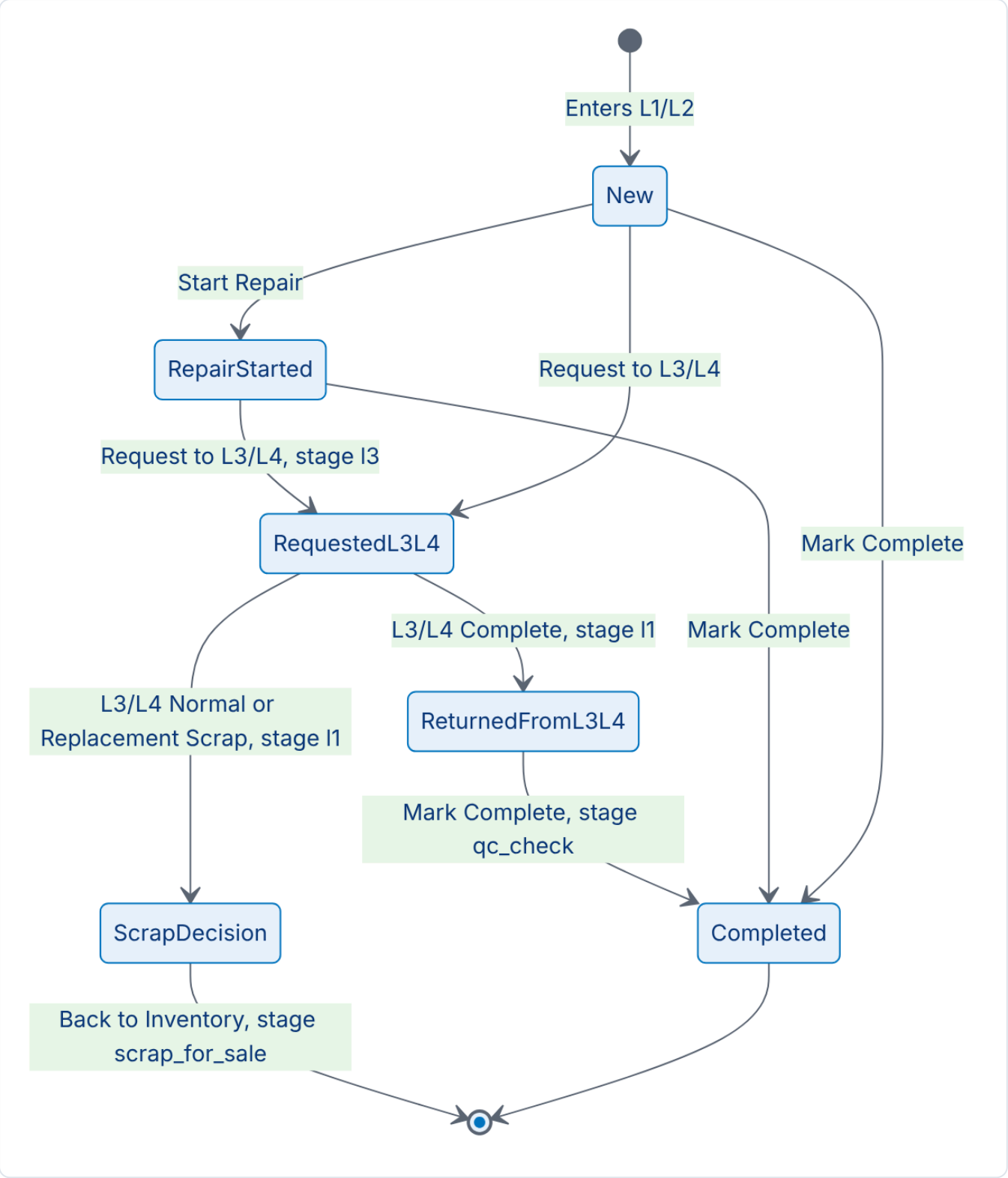
5. Flags and status fields beside the stage

Field	Values	Set by	Cleared by
`is_blocked`	true / false	`block_tags.py::create_block`	`_release_items` ; sale from block
`flag_*` (Urgent flag)	4 required fields	`stock.py::stock_flag_save`	Remove action; automatically on entry to `stock_in` (`models/device.py::_clear_flag_on_stock_in`)
`return_status`	true / false	Any return or credit note	`return_stock_complete` ; last Return deleted
`tag_return_status`	"Return for Repair", "Replaced by ", "Return for Credit Note"	`process_return` , `process_credit_note`	Last Return deleted
`final_qc_status`	pass / fail	Final QC decision	Edit (revert)
`grade`	A, B, C, D, E, scrap (`DeviceGrade`)	IQC form, Customise, Final QC Pass, Back to Inventory (scrap)	—
`scrap_verified`	true / false	`scrap.py::verify_scrap`	—
`sub_lot_number`	free text	GRN "Yes As-Is", transfers As-Is Lot, Customise	Customise Stock Type Finish Good / In Progress

6. Sub-state machines

6.1 L1/L2 and L3/L4 repair status

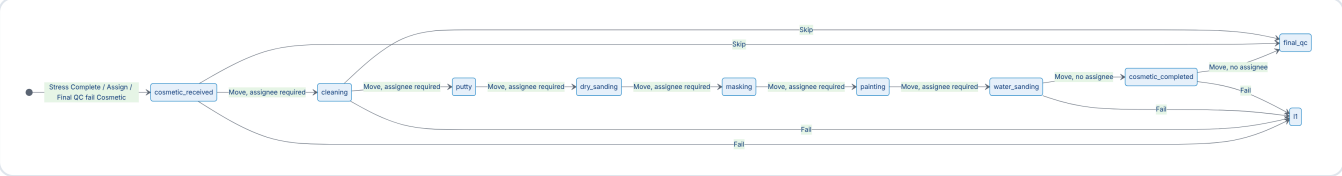
`devices.l1l2_status` and `devices.l34_status` (`models/device.py`). Both reset when a tag enters `l1` / `l2` from any other stage (`reset_repair_cycle_on_entry`).



`l34_status` values: blank → "Repair Started" → "Completed" or "Normal Scrap" / "Replacement Scrap".

6.2 Cosmetic chain

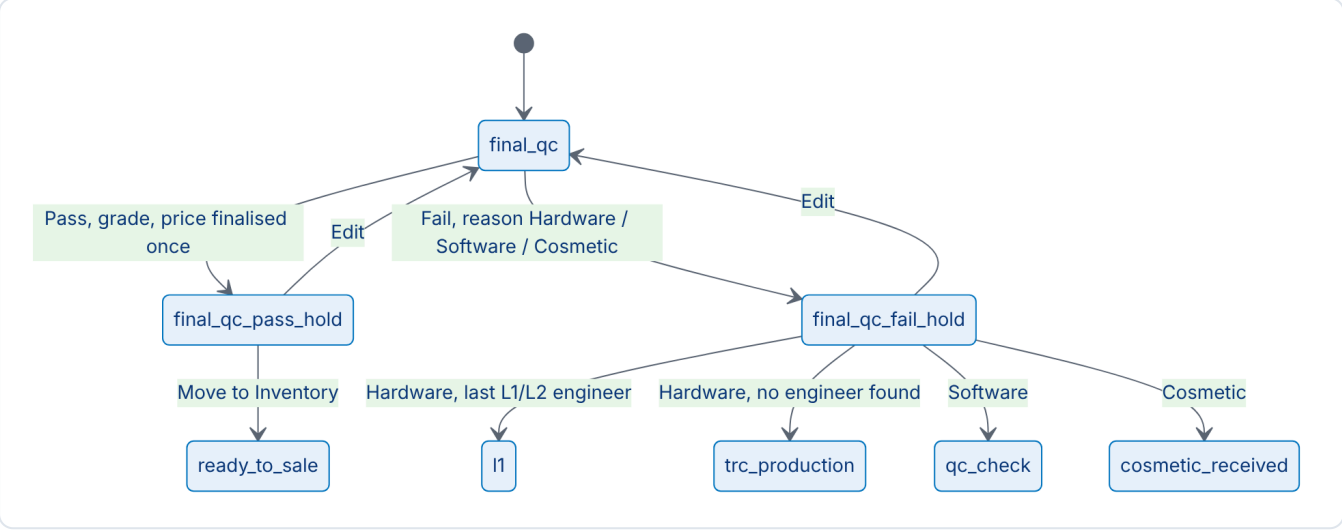
Order is fixed in `routers/cosmetic.py::COSMETIC_PIPELINE` and `NEXT_COSMETIC`.



Stage	WorkID code	Permission module	Fail allowed	Bulk Assign
Cosmetic Received	`recv`	`cosmetic_received`	Yes	Yes (also moves to Cleaning)
Cleaning	`clean`	`cosmetic_cleaning`	Yes	Yes (reassign only)
Putty	`putty`	`cosmetic_putty`	No	Yes (reassign only)
Dry Sanding	`drysd`	`cosmetic_dry_sanding`	No	Yes (reassign only)
Masking	`mask`	`cosmetic_masking`	No	Yes (reassign only)
Painting	`paint`	`cosmetic_painting`	No	Yes (reassign only)
Water Sanding	`water`	`cosmetic_water_sanding`	Yes	Yes (reassign only)
Cosmetic Completed	`comp`	`cosmetic_completed`	Yes	No

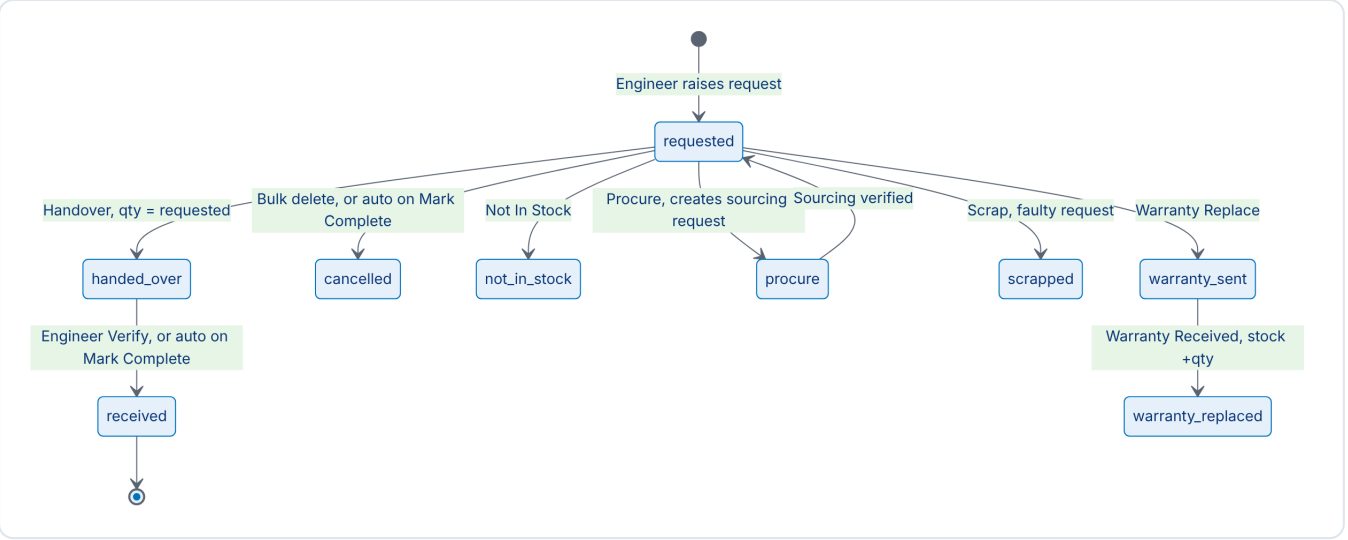
Skip to Final QC is allowed from any cosmetic stage (`advance_stage`, `target=final_qc`). Flow Data rows (`models/cosmetic_flow.py::CosmeticFlowRow`) name one user per stage; when the mover sits in a row, the next user is picked automatically.

6.3 Final QC holds



6.4 Part request and PNA

`part_requests.status` (`models/part_request.py::PartRequest`). Actions in `routers/part_requests.py` and `routers/repair.py`.



Action	Endpoint	Role	Guard
Raise (single / multi / bulk)	`create_part_request`, `create_part_requests_batch`, `repair.py::bulk_part_request`	`admin`, `inventory_manager`, `l1/l2/l3_engineer` (bulk: any user)	Bulk skips tags with a live request for the same part
Handover	`handover_part`	`admin`, `spare_parts_manager`	None on current status (single); bulk only from `requested`
Verify receiving	`validate_receiving`	Any user	None
Not In Stock / Procure / Scrap / Warranty Replace	`not_in_stock`, `procure_part`, `scrap_part_request`, `warranty_replace`	`admin`, `spare_parts_manager`	None on current status
Warranty Received	`warranty_received`	`admin`, `spare_parts_manager`	Must be `warranty_sent`

Sourcing request (`PartSourcingRequest.status`): `open` → `closed` (Sales Manager, `close_sourcing`) → verified (Spare Parts Manager, `verify_sourcing`, needs `closed`; adds stock via `SparePartsLedger` IN). Production requests from Part Estimation (`source = production`) are confirmed instead (`confirm_production_sourcing`).

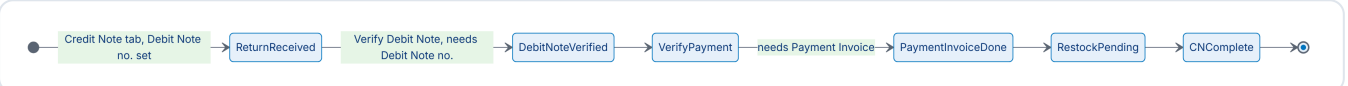
PNA (`models/pna_part.py::DevicePNAPart`): one row per tag + part. Mark sets `is_active` = true; clear sets false and keeps the row (`mark_part_pna`, `clear_all_pna`; any signed-in user). While any PNA row is active, L1/L2 Mark Complete is refused (T23).

6.5 Return approval

`returns.approval_status`: `pending` → `approved` (tag to `iqc` or `scrapped`) or `rejected` (no stage change). Only `admin` and `sales_manager` (`routers/sales.py::MANAGER_ROLES`).

6.6 Credit note

`returns.cn_stage` (`models/sales.py::CN_STAGES`). It never changes the device stage.



- Entry: `routers/sales.py::process_credit_note` (sales roles + `returns` add). It creates a Return and placeholder ₹0 sale if the tag has none. Re-submitting never moves a record back.
- Advance: one step at a time, `credit_note_advance` or `credit_note_bulk_verify` (`credit_note` edit permission), rules in `_try_advance_return`.

6.7 Block item, WorkOrder and delivery

Machine	States	Source
Block item	`blocked` → `released` (by Release or by sale from block)	`models/block_tags.py::BlockItemStatus`
WorkOrder	`pending` → `in_progress` (L3/L4 start) → `completed`	`models/work_order.py`
Sale delivery	`pending` → `dispatched` (Gate Pass exit) → `delivered`	`models/sales.py::Sale.delivery_status`, `routers/gate_pass.py::confirm_exit`
Return Stock repair cost	Verify records repair + labour cost, Complete re-sells	`routers/stock.py::return_stock_verify`, `return_stock_complete`

7. Invariants

These hold in the code today, with the exceptions named.

#	Invariant	Enforced by	Known exceptions
I1	A tag reaches `sold` only with a Sale row	`create_sale`, `process_return` (Replace Now), `return_stock_complete`; `sold` is not in `DROPDOWN_STAGES`; Customise refuses `sold`	`repair.py::move_device` and `api_v1/devices.py::move_device_stage` accept `sold` if `ready_to_sale → sold` is in `allowed_transitions` (Q3). `process_return` makes a placeholder sale for old orphans
I2	A blocked tag cannot be sold, except from its own block	`control_engine.validate_sale_allowed`, `create_sale::_check_sellable`	—
I3	A seller can only sell tags of their own entity	`create_sale::_check_sellable`	`return_stock_complete` and Replace Now do not check entity
I4	One return per sale	`process_return`	External Tag and Credit Note create their own placeholder sale
I5	Every stage move writes a StageMovement	Each endpoint	`stock.py::advance_grn_to_iqc`, `buckets.py::create_bucket` (Q12)
I6	Entry into L1/L2 from another stage starts a fresh repair cycle	`models/device.py::reset_repair_cycle_on_entry`; Stress/Cosmetic fail set it directly	`repair.py::move_device` does not call it
I7	A tag with active PNA parts cannot be Marked Complete in L1/L2	`l1l2_complete_to_stress`	Legacy `complete_repair` and Customise moves do not check PNA
I8	The Final QC price is finalised once per tag	`advance_stage` checks for an earlier "price finalized" StageMovement	—
I9	Entering `stock_in` clears the urgent flag	SQLAlchemy listener `_clear_flag_on_stock_in`	Bulk `update()` statements bypass ORM listeners

#	Invariant	Enforced by	Known exceptions
I10	Credit Note stages only move forward, one at a time	`_try_advance_return`	Delete removes the record
I11	Leaving L1/L2 closes the L1/L2 WorkIDs	`_close_l1l2_work_orders`, `close_open_work_orders`	Stress Complete/Fail do not close the open `STRS-` WorkID (Q10)

8. Allowed-transitions table

The live table is edited by admins on `/stage-control` (`routers/stage_control.py::add_transition`, `remove_transition`). Three seed lists exist and differ:

Source	Includes cosmetic_received / cosmetic_completed / putty?	Notes
`models/stage_control.py::DEFAULT_TRANSITIONS`	No	Has `qc_check → cleaning`, `final_qc → cleaning`
`upgrade_db.py` (CORRECT_TRANSITIONS)	No	Smaller list
`db_validator.py::CANONICAL_TRANSITIONS`	cosmetic_received and cosmetic_completed yes; putty no	Has fail moves to `l1`

None of the three includes `putty`, `final_qc_pass_hold`, `final_qc_fail_hold`, `scrap_for_sale` or `grn`.

9. Open questions / gaps

#	Gap	Code fact	Business view	Suggested owner
Q1	Sale from any stage	`create_sale` sells from any stage, including `sold`, `scrapped` and in-repair stages (decision dated 19 Oct 2026 in `control_engine.validate_sale_allowed`). A sold tag can be sold again	Sales Executive sells **Ready for Sale** items	Director / Sales
Q2	`returned` stage unused; reject leaves tag in Stock In	`process_return` moves the tag to `stock_in` at once; approve moves to `iqc` / `scrapped`; reject leaves it at `stock_in` though the code comment says "Device stays sold"	Returns: return, paid repair, replacement, credit note	Sales Manager
Q3	Transition rules are weak	Three seed lists differ (section 8); most workflow screens skip `validate_transition`; Manual Move and API can set `sold`	Clear, enforced stage flow	Architect
Q4	Roles in code ≠ 12 business roles	No built-in Production Manager, Final QC Inspector, Accounts or TRC Manager role. Production Manager = `inventory_manager` (`routers/buckets.py::production_manager`). Final QC = anyone with `cosmetic_finalqc` edit. Custom roles work through the permission matrix (`auth/dependencies.py::require_roles`)	12 distinct roles	Admin
Q5	Many moves have no stage guard and only need a login	`l1l2_complete_to_stress`, `back_to_production`, `l3l4_complete`, `l3l4_scrap`, `stress_fail_assign`, `stress_complete_to_paint`, `move_to_stock`, `send_to_cosmetic`, `scrap_move_to_inventory`	Only the owner role moves a tag forward	Architect

#	Gap	Code fact	Business view	Suggested owner
Q6	Scrap sale does not move tags	`scrap_for_sale.py::sell_scrap_for_sale` marks the batch sold; tags stay at `scrap_for_sale`. Two scrap stages (`scrapped` , `scrap_for_sale`)	Tags scrapped handled by Inventory Manager	Inventory
Q7	Customise bypass depends on a form field	`iqc_bulk_apply_grade_type` skips transition checks when the posted `return_to` is `/devices` or `/stock`	—	Architect
Q8	IQC Inspector cannot run GRN in code	`routers/grn.py` allows only `admin` , `inventory_manager`	IQC Inspector owns GRN, Lot, IQC	Admin
Q9	Two grade scales	`QCCheck.grade` A/B/C/D/S (`models/qc.py`) vs `DeviceGrade` A–E/scrap	Final QC assigns grading	Final QC
Q10	Open `STRS-` WorkID after Stress	Stress Complete/Fail create new WorkIDs but do not close the `STRS-` one raised by Mark Complete	Productivity reporting	Production
Q11	Block create trusts the client	`create_block` does not re-check `ready_to_sale` ; no audit on block/release; model comment mentions a "Done" status the enum lacks	Block tags for any staff	Inventory / Sales
Q12	Silent stage moves	`advance_grn_to_iqc` and `create_bucket` change stage without StageMovement; `create_bucket` moves any stage to `stock_in`	Full history	Architect
Q13	Rentals	No rental flow found in sales, dispatch or gate pass	Sales Executive handles rentals	Sales

PART B

Business Rules & SOPs

1. Purpose

This document lists the business rules the OxyPC Inventory Lifecycle app enforces today, and a short daily routine (SOP) for each of the 12 business roles.

- Every rule cites the file and function it comes from. If a rule is not in the code, it is not in this list.
- "Role" is the business persona. Where the code checks a different role, the code role is shown in brackets.
- Where the business view and the code disagree, the gap is listed in section 4.

Stage names and transitions are in the Device Lifecycle State Machine.

1.1 Business roles and the code roles behind them

#	Business role	Built-in code role(s) today	Note
1	IQC Inspector	`iqc_inspector` ("IQC Handler")	Cannot open `/grn` unless the permission matrix grants it
2	Inventory Manager	`inventory_manager`	Also acts as Production Manager in code
3	Production Manager	`inventory_manager`, or a custom role `production_manager`	No built-in role (`routers/buckets.py::production_manager`)
4	Repair Engineer	`l1_engineer` ("L1/L2 Engineer"), `l2_engineer`, `l3_engineer` ("L3/L4 Engineer")	
5	QC Inspector	`qc_inspector` ("QC Handler")	
6	Cosmetic Lead	`cosmetic_manager`	Single-stage custom roles also exist
7	Final QC Inspector	Any role with `cosmetic_finalqc` edit permission	No built-in role
8	Spare Parts Manager	`spare_parts_manager` ("Parts Manager")	
9	Sales Executive	`sales` ("Sourcing Sales"), `telecaller`, `sales_manager`	
10	Accounts	None; Accounts pages allow `admin`, `inventory_manager`, `sales_manager`	`routers/accounts.py::FINANCE_ROLES`
11	Managers / Admin	`admin`	Admin passes every role check
12	TRC Manager	Custom role `trc_manager`	Sees the full repair queue (`routers/repair.py::FULL_QUEUE_ROLES`)

Custom (admin-created) roles pass any non-admin-only `require_roles` gate and are then limited by the Module Permission matrix (`auth/dependencies.py::require_roles`).

2. Business rules

2.1 Intake: Lot, GRN and IQC

ID	Rule	Source	Role
BR-01	A tag number (barcode) must be unique; IQC refuses a barcode that already exists.	`routers/iqc.py::iqc_create`	IQC Inspector
BR-02	An IQC entry with no lot is filed under the system "Unassigned" lot.	`routers/iqc.py::iqc_create`, `utils/lot_helpers.py::get_or_create_unassigned_lot`	IQC Inspector
BR-03	An IQC entry that carries a GRN number goes straight to Stock In; without one it stays at IQC.	`routers/iqc.py::iqc_create`	IQC Inspector
BR-04	The device price defaults to the lot line item's unit price, else the lot average (buying price ÷ qty).	`routers/iqc.py::iqc_create`	IQC Inspector
BR-05	Tags registered on a lot start at GRN Receipt and move to IQC with "Advance GRN to IQC".	`routers/stock.py::register_device_barcode`, `advance_grn_to_iqc`	IQC Inspector
BR-06	Mapping a GRN moves only tags at IQC to Stock In; tags in other stages just receive the GRN number.	`routers/grn.py::grn_map`	Inventory Manager (code: `admin`, `inventory_manager`)
BR-07	A tag with no entity gets "OxyPC Computers" when its GRN is mapped; an existing entity is never overwritten.	`routers/grn.py::grn_map`	Inventory Manager
BR-08	Validating a Plant GRN auto-maps every IQC-stage tag of the linked lot to Stock In.	`routers/grn.py::grn_validate`	Inventory Manager
BR-09	A lot GRN records expected vs received quantity and warns on a mismatch; the GRN number is auto-generated if blank.	`routers/grn.py::submit_grn`	Inventory Manager
BR-10	"Pending for TRC" bulk move sends tags back to IQC with the GRN cleared ("Sending for Selling") or to Stock In ("Sold to OxyPC"); tags whose move is not an allowed transition are skipped.	`routers/grn.py::grn_bulk_move_to_trc`	Inventory Manager
BR-11	"Yes As-Is" writes one Sub-Lot Number (and optional Assigned To) on every tag mapped to that GRN.	`routers/grn.py::grn_as_is_lot`	Inventory Manager

2.2 Stock and floor movement

ID	Rule	Source	Role
BR-12	Only tags at Stock In can be moved to TRC Production or to L1/L2.	`routers/stock.py::move_to_trc`, `_move_device_to_l1l2`	Inventory Manager
BR-13	A tag moved to L1/L2 from Stock In lands unassigned (Pick This), and any stale L1 WorkID is closed first.	`routers/stock.py::_move_device_to_l1l2`	Inventory Manager
BR-14	"Move to Production" on a bucket moves only its Stock In tags, except customer-return buckets, which move all their tags.	`routers/buckets.py::_move_bucket_devices_to_trc`	Inventory Manager
BR-15	A bucket name is reused case-insensitively; creating a bucket sets its tags to Stock In.	`routers/buckets.py::create_bucket`	Inventory Manager

ID	Rule	Source	Role
BR-16	An urgent flag needs all four fields: Authorized By, Request Date, Target Date and Reason.	`routers/stock.py::stock_flag_save`	Inventory Manager
BR-17	The urgent flag clears itself whenever a tag moves into Stock In.	`models/device.py::_clear_flag_on_stock_in`	System
BR-18	A transfer (Move Item / Bucket / Lot) records the assignment and location only; it never changes the stage.	`routers/transfers.py::create_transfer`, `_move_devices_bulk`	Inventory Manager
BR-19	Every transfer needs an employee to assign to and creates an `asgn` WorkID plus a notification.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-20	A transfer of type Ready for Sale (shown as "Finish Good") or As-Is Lot stamps the employee as the tag's Assigned To.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-21	A transfer of type Scrap for Sale creates one Scrap ID batch per submission.	`routers/transfers.py::_maybe_create_scrap_batch`	Inventory Manager
BR-22	A new Location ID resets the tag's Bin Location and writes a location log row.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-23	The Customise modal can never set a tag to Sold; selling must go through New Sale.	`routers/iqc.py::iqc_bulk_apply_grade_type`	Inventory Manager
BR-24	Customise stage moves skip the allowed-transition check for admin and when used from All Inventory or Inventory Manager pages.	`routers/iqc.py::iqc_bulk_apply_grade_type`	Inventory Manager
BR-25	ReGRN (Entity Movement) moves a sold tag back to Ready to Sale under the new GRN, invoice and entity.	`routers/entity_movement.py` (change entities)	Inventory Manager / TRC Manager

2.3 Production and repair

ID	Rule	Source	Role
BR-26	Production assignment has three departments: L1/L2 Repair → L1/L2, Stress Test → Stress Test, Cosmetic Repair → Cosmetic Received.	`routers/buckets.py::DEPT_TO_STAGE`, `_apply_department_move`	Production Manager
BR-27	Reassigning a tag closes the previous person's open WorkID and opens a new one at the same moment.	`services/work_order_close.py::close_open_work_orders`	Production Manager
BR-28	Assigning a Stress Test engineer reuses the open, unassigned STRS- WorkID instead of creating a second one.	`services/work_order_close.py::reuse_open_stress_work_order`	Production Manager
BR-29	Change Engineer can move a tag from any stage to L1/L2, L3/L4, Stress Test, Cosmetic, Final QC, Stock In or TRC Production and assign any active user.	`routers/stock.py::change_engineer`, `CHANGE_ENGINEER_STAGES`	Production Manager
BR-30	Any tag entering L1/L2 from another stage starts a fresh repair cycle (status New, L3/L4 status cleared).	`models/device.py::reset_repair_cycle_on_entry`	System
BR-31	"Pick This" lets the clicking engineer take a tag from anyone; all open WorkIDs close and a new one opens.	`routers/repair.py::l1_pick`, `l3l4_pick`	Repair Engineer
BR-32	Engineers see their own tags plus the unclaimed pool; admin, TRC Manager, Inventory Manager and Production Manager see the full queue.	`routers/repair.py::FULL_QUEUE_ROLES`	Repair Engineer / supervisors

ID	Rule	Source	Role
BR-33	Request to L3/L4 moves the tag to L3/L4 and creates an L3L4- WorkID for the chosen engineer; the L1/L2 WorkID stays open.	`routers/repair.py::request_l3l4`	Repair Engineer (L1/L2)
BR-34	L3/L4 Complete and L3/L4 Scrap both send the tag back to L1/L2 for the next action.	`routers/repair.py::l3l4_complete`, `l3l4_scrap`	Repair Engineer (L3/L4)
BR-35	L3/L4 scrap must be "Normal Scrap" or "Replacement Scrap".	`routers/repair.py::l3l4_scrap`	Repair Engineer (L3/L4)
BR-36	After an L3/L4 scrap decision, "Back to Inventory" sets grade Scrap and moves the tag to Scrap for Sale (Tags Scrapped page).	`routers/repair.py::back_to_production`	Repair Engineer
BR-37	L1/L2 Mark Complete is refused while the tag has any active PNA part.	`routers/repair.py::l1l2_complete_to_stress`	Repair Engineer
BR-38	Mark Complete auto-verifies handed-over parts (cost counted) and auto-cancels parts never handed over.	`routers/repair.py::_mark_complete_open_parts`, `l1l2_complete_to_stress`	Repair Engineer
BR-39	Mark Complete always moves the tag to Stress Test and raises an STRS- WorkID; the Stress engineer is optional.	`routers/repair.py::l1l2_complete_to_stress`	Repair Engineer
BR-40	In the legacy Complete Job panel, a repair whose total cost reaches the expected sale value is auto-scrapped; at 70% the engineer is warned.	`services/cost_engine.py::check_scrap_decision`, `routers/repair.py::complete_repair`	System
BR-41	L2 repair needs an earlier L1 attempt and L3 needs an L2 attempt (legacy Start Repair).	`services/control_engine.py::validate_repair_level`	Repair Engineer
BR-42	The L2 stage is retired; every escalation or fail lands in L1/L2.	`routers/repair.py::complete_repair`, `routers/stress_api.py::stress_fail_assign`	System
BR-43	Part Estimation costs a lot's IQC-required parts; "Download & Attach Estimate" raises a whole-lot production sourcing request.	`routers/part_estimation.py` (module docstring)	Production Manager

2.4 QC (Stress Test)

ID	Rule	Source	Role
BR-44	The Stress Test checklist has 27 fixed items; only Touch Screen may be marked N/A.	`models/stress.py::STRESS_CHECKLIST_ITEMS`, `STRESS_OPTIONAL_ITEMS`	QC Inspector
BR-45	Stress Fail must assign an L1 or L2 engineer and sends the tag to L1/L2 with the failed items saved as Stress Notes.	`routers/stress_api.py::stress_fail_assign`	QC Inspector
BR-46	Stress Complete sends the tag to Cosmetic Received and clears old Stress Notes; an assignee is optional.	`routers/stress_api.py::stress_complete_to_paint`	QC Inspector
BR-47	Each Stress result gives the tester a completed WorkID and saves one checklist report (with PDF) per visit.	`routers/stress_api.py::_record_checklist_result`, `stress_fail_assign`	QC Inspector
BR-48	On the legacy QC form, a score of 70/100 or more passes, and the 3rd fail is noted "recommend scrap".	`routers/qc.py::qc_submit`, `QC_FAIL_SCRAP_THRESHOLD`	QC Inspector

2.5 Cosmetic

ID	Rule	Source	Role
BR-49	The cosmetic chain runs in this order: Cosmetic Received, Cleaning, Putty, Dry Sanding, Masking, Painting, Water Sanding, Cosmetic Completed, Final QC.	`routers/cosmetic.py::COSMETIC_PIPELINE`, `NEXT_COSMETIC`	Cosmetic Lead
BR-50	Every cosmetic Move needs an assignee, except a move that lands on Cosmetic Completed or Final QC.	`routers/cosmetic.py::advance_stage`	Cosmetic Lead
BR-51	If the mover appears in a saved Flow Data row, the next stage's user is assigned automatically.	`routers/cosmetic.py::_resolve_flow_next_user`	Cosmetic Lead
BR-52	A tag can skip the rest of the cosmetic chain and go straight to Final QC from any cosmetic stage.	`routers/cosmetic.py::advance_stage`	Cosmetic Lead
BR-53	Cosmetic Fail is allowed only at Cosmetic Received, Cleaning, Water Sanding and Cosmetic Completed, and must assign an L1/L2 engineer.	`routers/cosmetic.py::cosmetic_fail_assign`	Cosmetic Lead
BR-54	Bulk Assign is for admin and Cosmetic Manager only; at Cosmetic Received it also moves tags to Cleaning.	`routers/cosmetic.py::bulk_assign`	Cosmetic Lead
BR-55	Each cosmetic page needs its own edit permission (for example `cosmetic_painting`).	`routers/cosmetic.py::PERM_MODULE_BY_STAGE`	Cosmetic Lead
BR-56	Only admin and Cosmetic Manager can create or edit Flow Data rows.	`models/cosmetic_flow.py::CosmeticFlowRow`, `routers/cosmetic.py::flow_data_allowed`	Cosmetic Lead

2.6 Final QC and grading

ID	Rule	Source	Role
BR-57	Final QC decisions need edit permission on `cosmetic_finalqc`.	`routers/cosmetic.py::advance_stage`, `fqc_move_passed`, `fqc_move_failed`	Final QC Inspector
BR-58	Final QC Pass sets the grade (A, B, C, D, E or Scrap) and holds the tag in "Pass Hold".	`routers/cosmetic.py::advance_stage`, `models/device.py::DeviceGrade`	Final QC Inspector
BR-59	On the first Final QC Pass, the device price becomes buying price + parts consumed + changed parts received; later passes do not add again.	`routers/cosmetic.py::advance_stage`	System
BR-60	Verified RAM and hard-drive changes are written onto the tag's spec at Final QC Pass.	`services/parts_changed.py::apply_spec_changes_on_pass`	System
BR-61	Final QC Fail needs a reason; the tag waits in "Fail Hold" until someone clicks Move.	`routers/cosmetic.py::advance_stage`	Final QC Inspector
BR-62	The fail reason routes the tag: Hardware → L1/L2, Software → Stress Test, Cosmetic → Cosmetic Received, each to the last person who worked that step.	`routers/cosmetic.py::FAIL_REASON_TO_DEPT`, `_resolve_fail_engineer`, `fqc_move_failed`	Final QC Inspector
BR-63	A Hardware fail with no engineer found goes to TRC Production (Tag Number Allocation).	`routers/cosmetic.py::fqc_move_failed`	Final QC Inspector
BR-64	"Move to Inventory" moves Pass Hold tags to Ready to Sale.	`routers/cosmetic.py::fqc_move_passed`	Final QC Inspector → Inventory Manager

ID	Rule	Source	Role
BR-65	"Edit" returns a Pass Hold or Fail Hold tag to Final QC and clears the decision.	`routers/cosmetic.py::fqc_edit_revert`	Final QC Inspector
BR-66	Touch Type is saved on both Pass and Fail; a blank value shows as Non-Touch.	`routers/cosmetic.py::advance_stage`, `models/device.py::touch_type`	Final QC Inspector

2.7 Spare parts

ID	Rule	Source	Role
BR-67	Engineers raise part requests as New, Replace or Downgrade; the request is linked to the tag's open WorkID.	`routers/part_requests.py::create_part_request`	Repair Engineer
BR-68	Bulk Part Request raises one request per L1/L2 tag that needs the part and skips tags that already have a live request for it.	`routers/repair.py::bulk_part_request`	Repair Engineer
BR-69	The handover quantity always equals the requested quantity; it cannot be edited.	`routers/part_requests.py::handover_part`, `bulk_request_action`	Spare Parts Manager
BR-70	Bulk Handover acts only on requests still at "requested".	`routers/part_requests.py::bulk_request_action`	Spare Parts Manager
BR-71	Deleting a part request is a soft close ("cancelled"); handed-over or received requests cannot be cancelled.	`routers/part_requests.py::bulk_request_action`	Spare Parts Manager
BR-72	"Procure" creates a sourcing request that the Sales Manager closes in the CRM.	`routers/part_requests.py::procure_part`, `close_sourcing`	Spare Parts Manager / Sales Manager
BR-73	A sourcing request can be verified only after its deal is closed; verifying adds the sourced quantity to stock and reopens the part request.	`routers/part_requests.py::verify_sourcing`	Spare Parts Manager
BR-74	A faulty part under supplier warranty goes Warranty Sent → Warranty Replaced, and the replacement goes back into stock.	`routers/part_requests.py::warranty_replace`, `warranty_received`	Spare Parts Manager
BR-75	Any signed-in user can mark a part PNA on a tag; clearing keeps the history.	`routers/part_requests.py::mark_part_pna`, `models/pna_part.py`	Repair Engineer
BR-76	Parts GRN (As New) records PO, invoice, e-way bill and vehicle details and adds line quantities to Part Master stock.	`routers/parts_grn.py::grn_create`	Spare Parts Manager
BR-77	A harvest part (from an old device) is recorded as an internal Parts GRN ("Harvest") and added to Part Master with source "harvest".	`routers/parts_grn.py::harvest_part`	Spare Parts Manager

2.8 Sales and dispatch

ID	Rule	Source	Role
BR-78	A seller can only sell tags whose entity matches their own profile entity; a seller with no entity cannot sell.	`routers/sales.py::create_sale` (`_check_sellable`)	Sales Executive
BR-79	A blocked tag cannot be sold, except through its own block's Sell button.	`services/control_engine.py::validate_sale_allowed`, `routers/sales.py::create_sale`	Sales Executive

ID	Rule	Source	Role
BR-80	Since 19 Oct 2026 a tag can be sold from any stage.	`services/control_engine.py::validate_sale_allowed`	Sales Executive
BR-81	A sale below the device's total cost is allowed but shows a below-cost warning.	`services/cost_engine.py::below_cost_warning_for`	Sales Executive
BR-82	Sale numbers come from one sequence (SALE-0001…) and are drawn only for tags that will actually sell.	`routers/sales.py::create_sale`	System
BR-83	The selling company is matched to the tag's entity and snapshotted on the sale, so invoices never change later.	`routers/sales.py::create_sale`, `routers/invoices.py::_sale_company_snapshot`	System
BR-84	A sale's invoice number replaces the tag's purchase invoice number.	`routers/sales.py::create_sale`	Sales Executive
BR-85	Warranty expiry = sale date + the warranty type's days.	`routers/sales.py::create_sale`	Sales Executive
BR-86	Only admin can delete sales or change a sale date; referenced sales are skipped and the tag returns to the stage it was sold from.	`routers/sales.py::bulk_delete_sales`, `change_sale_date`	Admin
BR-87	Block Tags hold Ready to Sale tags for a staff member or customer; the Block ID is `DDMMYYYY-CONTAINER-0001`, counting per date and container.	`routers/block_tags.py::create_block`, `_next_block_id`, `search_tags`	Inventory Manager, Sales Executive
BR-88	Setting a Ready to Sale price needs the "Set Price" pricing permission.	`routers/sales.py::_role_can_set_price`	Sales Executive / Manager
BR-89	The Gate Pass lists every sale with its Delivery Challan; confirming exit marks the sale "dispatched" and skips ones already dispatched.	`routers/gate_pass.py::confirm_exit`, `bulk_confirm_exit`	Sales Executive / Inventory Manager
BR-90	A telecaller's dispatch request is approved by the Sales Manager on Ready to Dispatch.	`routers/dispatch.py::approve_dispatch`	Sales Executive / Sales Manager

2.9 Returns and credit notes

ID	Rule	Source	Role
BR-91	An internal return needs a sale on file, a reason, a condition and a "Received By" person.	`routers/sales.py::process_return`	Sales Executive
BR-92	Only one return is allowed per sale.	`routers/sales.py::process_return`	Sales Executive
BR-93	A returned tag moves to Stock In at once; the return waits for manager approval.	`routers/sales.py::process_return`	Sales Executive
BR-94	Replace Now sells a Ready to Sale tag at ₹0 to the same customer; it cannot be the returned tag.	`routers/sales.py::process_return`	Sales Executive
BR-95	Only admin or Sales Manager can approve or reject a return; approval sends the tag to IQC (restock) or Scrapped.	`routers/sales.py::approve_return`, `reject_return`, `MANAGER_ROLES`	Managers
BR-96	Warranty status at return is computed by the server from the sale, never typed in.	`routers/sales.py::process_return`	System
BR-97	An External Tag return creates the tag if unknown (lot EXT-RETURNS) and a ₹0 placeholder sale, and records the paid-repair amount.	`routers/sales.py::process_external_return`	Sales Executive

ID	Rule	Source	Role
BR-98	Return Stock "Verify" records repair and labour cost; "Complete" re-sells the tag to the original customer under a new sale number.	`routers/stock.py::return_stock_verify` , `return_stock_complete`	Inventory Manager
BR-99	A credit note needs a Debit Note number and a "Received By" person, and marks the tag "Return for Credit Note".	`routers/sales.py::process_credit_note`	Sales Executive
BR-100	Credit notes move forward one step at a time through six stages; "Payment Invoice Done" needs a payment invoice number.	`routers/sales.py::_try_advance_return` , `models/sales.py::CN_STAGES`	Accounts / Sales Manager (`credit_note` edit)

2.10 Accounts

ID	Rule	Source	Role
BR-101	Supplier payments and customer receipts can be added only by admin, Inventory Manager or Sales Manager, and the amount must be above zero.	`routers/accounts.py::create_supplier_payment` , `FINANCE_ROLES`	Accounts
BR-102	The Tax Invoice and Delivery Challan print from the sale's frozen company snapshot.	`routers/invoices.py` (`/print` , `/waybill`)	Accounts
BR-103	Lot P&L and other financial reports need the financial report roles.	`routers/reports.py::_require_financials`	Accounts / Managers
BR-104	Daily Stock reports are available as overall, tag, location and daily workbook exports.	`routers/reports.py::daily_stock_report` and export routes	Accounts / TRC Manager

2.11 Warranty

ID	Rule	Source	Role
BR-105	A warranty can be set only on a tag that has a sale; the expiry is counted from the sale date.	`routers/extended_warranty.py::extended_warranty_set`	Inventory Manager / TRC Manager
BR-106	An extension date must be later than the current Warranty Stops date; the added days are kept as a running total and noted on the sale.	`routers/extended_warranty.py::extended_warranty_update`	Inventory Manager / TRC Manager

2.12 Scrap

ID	Rule	Source	Role
BR-107	The Tags Scrapped page lists every tag at Scrap for Sale, whatever its grade.	`routers/scrap.py::scrap_products`	Inventory Manager
BR-108	Verifying a scrapped tag's quantity enables its Sell button.	`routers/scrap.py::verify_scrap`	Inventory Manager
BR-109	A Replacement Scrap records which tag replaced it.	`routers/scrap.py::set_scrap_replacement` , `routers/stock.py::scrap_replace_with_another`	Inventory Manager

2.13 Audits and traceability

ID	Rule	Source	Role
BR-110	A physical audit records the stage and location seen against the system values; a tag can be audited only once per report date.	`routers/physical_audits.py::submit_audit`, `_already_audited`	TRC Manager
BR-111	Every stage move should write a StageMovement row; the previous open row gets an exit time.	`models/device.py::StageMovement`, all stage-move endpoints	System
BR-112	Only admin can edit the allowed-transitions table.	`routers/stage_control.py::add_transition`, `remove_transition`	Admin
BR-113	Each WorkID gets a Completed Date when the tag leaves that person's step.	`services/work_order_close.py`, `routers/repair.py::_close_l1l2_work_orders`	System

3. Standard operating procedures

Short daily routines. Screens are named as they appear in the app.

3.1 IQC Inspector

1. Open the lot (or create it) and register incoming tags, or use IQC → New.
2. Fill the IQC form: tag number, lot, specs, grade and location. Add the GRN number if it is known.
3. Run OxyQC on the device if the agent is used, and check the result reached IQC.
4. If tags were registered at GRN Receipt, click **Advance GRN to IQC**.
5. Before leaving, check the IQC list for tags without a GRN number and hand them to the Inventory Manager for mapping.

3.2 Inventory Manager

1. Map or validate GRNs on **GRN Post-IQC** so IQC tags move to Stock In.
2. Review **Inventory Manager (Stock)**: bucket new tags, set location, and flag urgent tags (all four flag fields).
3. Move tags to production: **Move to Production** (bucket) or **Move to L1/L2** (single or bulk). Only Stock In tags move.
4. Use **Transfers** for floor moves and assignments (Finish Good, As-Is Lot, Scrap for Sale).
5. On **Final QC → Devices Passed**, click **Move to Inventory** so tags reach Ready to Sale.
6. Work the **Return Stock** table: verify repair and labour cost, then Complete or rebucket.
7. Review **Tags Scrapped**: verify quantity and record replacements.
8. Create or release **Block Tags** when asked; set or extend warranties on **Extended Warranty**.

3.3 Production Manager

1. Open **TRC Production** and read the tiles (At You, L1/L2, L3/L4, Stress, Cosmetic, Final QC, Tags Returned).
2. Assign each bucket or tag to a department and engineer (L1/L2 Repair, Stress Test, Cosmetic Repair).
3. Clear the **Tag Number Allocation** queue, including Final QC Hardware fails with no engineer.
4. Use **Change Engineer** for absent staff; the old WorkID closes and a new one opens.
5. Review **PNA summary** and stuck tags; chase parts with the Spare Parts Manager.

6. For new lots, run **Part Estimation** and attach the estimate to raise the parts request.
7. Check **WorkID Status** for team output at the end of the day.

3.4 Repair Engineer (L1/L2 and L3/L4)

1. Open **L1/L2 Repair** (or **L3/L4**). Click **Pick This** on your next tag.
2. Click **Start Repair**.
3. Raise part requests from the tag; mark parts PNA if they cannot be sourced.
4. When a part is handed over, **Verify** it on the device page.
5. If deeper repair is needed, **Request to L3/L4** with a repair note.
6. L3/L4: finish with **Mark Complete** (choose the activity) or **Scrap** (Normal or Replacement).
7. L1/L2: clear all PNA marks, then **Mark Complete** to send the tag to Stress Test. After a scrap decision, use **Back to Inventory**.

3.5 QC Inspector (Stress Test)

1. Open **Stress Test** and take the next tag.
2. Install the OS and run the diagnostic or the stress runner if used.
3. Tick each of the 27 checklist items as Pass, Fail (or N/A for Touch Screen).
4. All pass: click **Complete**. The tag goes to Cosmetic Received.
5. Any fail: click **Fail**, pick an L1/L2 engineer and add a note. The tag goes back to L1/L2.

3.6 Cosmetic Lead

1. Open **Cosmetic & Paint** and check each stage tab.
2. On Cosmetic Received, **Bulk Assign** the day's tags to the Cleaning team (or Move one at a time).
3. Each stage worker clicks **Move** when done and picks the next person (or Flow Data picks them).
4. Use **Skip to Final QC** for tags that need no further cosmetic work.
5. If a tag has a hardware problem at Received, Cleaning, Water Sanding or Completed, click **Fail** and assign an L1/L2 engineer.
6. Keep Flow Data rows up to date when teams change.

3.7 Final QC Inspector

1. Open **Final QC** and take the next tag.
2. Run the manual diagnosis. Correct specs if needed (or use Revalidate IQC).
3. Pass: choose the grade, touch type and bucket name, add pass notes, submit.
4. Fail: choose the reason (Hardware, Software or Cosmetic), add notes, submit.
5. On **Devices Failed**, click **Move** or **Bulk Move** to send each tag back.
6. On **Devices Passed**, click **Move to Inventory** to hand tags to the Inventory Manager.

3.8 Spare Parts Manager

1. Open **Part Master → Requests**. Hand over parts in stock (bulk handover for many).
2. Mark **Not In Stock** or **Procure** for parts you cannot supply.
3. On **Sourcing Requests**, verify closed deals and confirm production requests.
4. Record new stock on **Parts GRN** (As New) with PO, invoice and e-way bill.

5. Record parts taken from old devices with **Add Harvest Part**.
6. Handle faulty parts: Scrap, or send under warranty and receive the replacement.

3.9 Sales Executive

1. Review **Ready to Sale**; check prices and blocked tags.
2. Block tags for a customer or colleague on **Block Tags** when asked.
3. Record sales on **New Sale** / **New Tag Sale**, or **Sell** from a block. Check entity warnings.
4. Print the invoice and Delivery Challan; confirm exit on **Gate Pass**.
5. Log customer returns on **Return New**: Internal Tag, External Tag (paid repair) or Credit Note. Use Replace Now for replacements.
6. Raise dispatch requests (telecallers) and review the sales report.

3.10 Accounts

1. Record **Customer Receipts** against sales and **Supplier Payments** against lots or POs.
2. Upload invoices on POs and quotes; complete payments.
3. Advance **Credit Notes** through Debit Note Verified, Verify Payment and Payment Invoice Done.
4. Check the **Sales** list and the **Daily Stock** report.

3.11 Managers / Admin

1. Read the **Dashboard** and key reports (Lot P&L, Sales, Stage Movement).
2. Approve or reject **Pending Returns**.
3. Handle **Tickets** raised by users.
4. Maintain users, roles, Master Data and the Module Permission matrix.
5. Admin only: correct sales (delete or change date) and edit **Stage Control** transitions.

3.12 TRC Manager

1. Review TRC Production, Inventory and Part Master to see the whole floor.
2. Check the **Daily Stock** report.
3. Run **Physical Audits** for the day's stages and locations; resolve mismatches.
4. Set or extend warranties on **Extended Warranty** when approved.
5. Escalate stuck tags (PNA, long WorkIDs) to the Production and Spare Parts Managers.

4. Open questions

#	Question	Code fact	Business view
OQ1	Should sales be limited to Ready to Sale?	Any stage can be sold, including a tag already Sold (`services/control_engine.py::validate_sale_allowed` , `routers/sales.py::create_sale`)	Sales Executive sells Ready for Sale items
OQ2	What happens to a rejected return?	The tag is already in Stock In and stays there; the `returned` stage is never used (`routers/sales.py::process_return` , `reject_return`)	Return, paid repair, replacement or credit note

#	Question	Code fact	Business view
OQ3	Who are Production Manager, Final QC Inspector, Accounts and TRC Manager in the system?	No built-in roles; Production Manager = `inventory_manager` ; Accounts pages hard-code `admin` , `inventory_manager` , `sales_manager` , so a custom Accounts role is refused (`routers/accounts.py::FINANCE_ROLES`)	12 distinct roles
OQ4	Should IQC Inspectors run GRN?	`/grn` allows only `admin` and `inventory_manager` (`routers/grn.py`)	IQC Inspector owns GRN, Lot, IQC
OQ5	Should stage moves check the current stage and the user's role?	Many moves accept any signed-in user and any current stage (for example `routers/repair.py::l1l2_complete_to_stress` , `routers/stress_api.py::stress_fail_assign` , `routers/stock.py::move_to_stock`)	Only the owning role moves a tag forward
OQ6	How are e-way bills generated for sales?	E-way bill fields exist only on inbound Parts GRN and lot GRN; no sale e-way bill flow found	Accounts generates e-way bills
OQ7	Where are rentals handled?	No rental flow found in sales, dispatch or gate pass	Sales Executive handles rentals
OQ8	Should a scrap sale move tags out of Scrap for Sale?	Selling a Scrap for Sale batch marks the batch sold only (`routers/scrap_for_sale.py::sell_scrap_for_sale`)	Inventory Manager handles scrapped tags
OQ9	Should harvest parts link to the source tag?	Harvest records a free-text lot number, not the device (`routers/parts_grn.py::harvest_part`)	Harvest parts come from old devices
OQ10	Which grade scale is official?	Device grade A–E/Scrap; legacy QC check grade A/B/C/D/S (`models/qc.py`)	Final QC assigns grading
OQ11	Should Block Tags be audited and stage-checked on the server?	No audit on create/release; Ready to Sale is checked only in the search box (`routers/block_tags.py::create_block`)	Block tags for any staff
OQ12	Should part request actions check the current status?	Handover, Not In Stock, Procure and Scrap do not check status on single rows (`routers/part_requests.py`)	Spare Parts Manager actions requests
OQ13	Is dispatch approval required before a telecaller sells?	The rule is in the dispatch module text; `create_sale` does not check it	—